

K-Means Clustering

Make_blobs: This function generates a random dataset of points (**features**) grouped into clusters (**blobs**), which are often used for testing clustering algorithms.

number of clusters (**centroids** or centers)

✓
7s



```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import make_blobs
```

```
[ ] x,y=make_blobs(n_samples=1000,centers=3,n_features=2)
```

```
✓ [4] x
0s
array([[ 2.71120103, -5.06078968],
       [-9.2489178 , -3.52011572],
       [-4.53913906,  3.72638877],
       ...,
       [-7.6481443 , -2.83959146],
       [-6.54914317,  4.56451787],
       [-5.14740306,  2.45040486]])

✓ y
0s
array([0, 1, 2, 0, 2, 0, 1, 0, 0, 0, 1, 1, 2, 1, 2, 1, 1, 1, 0, 1, 1, 1,
       2, 2, 1, 1, 2, 0, 0, 2, 1, 2, 0, 2, 2, 2, 1, 2, 0, 2, 0, 0, 0, 2,
       0, 0, 1, 0, 0, 2, 2, 2, 1, 1, 0, 0, 1, 0, 2, 1, 0, 2, 0, 0, 1, 2,
       0, 1, 1, 0, 1, 0, 0, 0, 2, 0, 2, 1, 1, 2, 0, 2, 1, 0, 1, 2, 1, 1,
       1, 0, 2, 0, 1, 0, 2, 2, 0, 1, 0, 1, 1, 0, 1, 0, 2, 0, 0, 0, 2, 1,
       2, 0, 0, 0, 1, 0, 2, 2, 1, 0, 0, 2, 1, 1, 2, 0, 2, 2, 1, 1, 0, 1,
       1, 2, 1, 1, 0, 1, 2, 1, 0, 2, 0, 0, 1, 2, 2, 1, 0, 2, 2, 2, 1, 2,
       0, 2, 0, 0, 1, 0, 1, 1, 2, 0, 1, 1, 2, 1, 0, 2, 1, 1, 1, 0, 0, 1,
       0, 1, 2, 2, 2, 2, 2, 2, 0, 2, 1, 0, 2, 0, 1, 2, 2, 2, 1, 0, 2, 1,
       2, 2, 1, 1, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 1, 1, 2, 2, 0, 0, 1, 1,
       1, 1, 0, 0, 1, 0, 0, 0, 2, 0, 2, 1, 2, 2, 1, 2, 1, 0, 1, 2, 0, 1,
       2, 0, 0, 1, 1, 2, 2, 0, 0, 0, 0, 0, 0, 1, 0, 2, 0, 2, 2, 1, 1, 1,
       1, 0, 2, 2, 1, 0, 1, 1, 1, 0, 1, 1, 2, 2, 1, 0, 2, 1, 0, 1, 1, 1,
       1, 0, 2, 0, 1, 1, 1, 0, 0, 1, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       2, 2, 2, 0, 1, 2, 0, 2, 2, 0, 0, 2, 1, 0, 2, 1, 1, 0, 0, 2, 1, 1,
       1, 2, 0, 0, 1, 2, 2, 2, 0, 2, 0, 1, 1, 1, 0, 0, 1, 0, 2, 0, 2, 2,
       2, 1, 0, 0, 1, 2, 1, 2, 1, 1, 0, 1, 2, 2, 1, 1, 2, 2, 1, 1, 0, 0,
       0, 1, 1, 2, 2, 1, 2, 0, 1, 2, 1, 2, 2, 1, 1, 0, 1, 1, 2, 0, 0, 2,
       1, 1, 2, 0, 0, 0, 0, 0, 2, 0, 2, 1, 0, 0, 0, 2, 2, 0, 2, 2, 2, 2,
       0, 2, 2, 0, 1, 0, 2, 2, 2, 1, 2, 0, 1, 2, 0, 2, 2, 2, 2, 0, 0, 0,
```

y is a labeled data it is generated only for **evaluation purposes** during testing.
Clustering algorithms like **KMeans** do not use **y** because they work without **labels**.

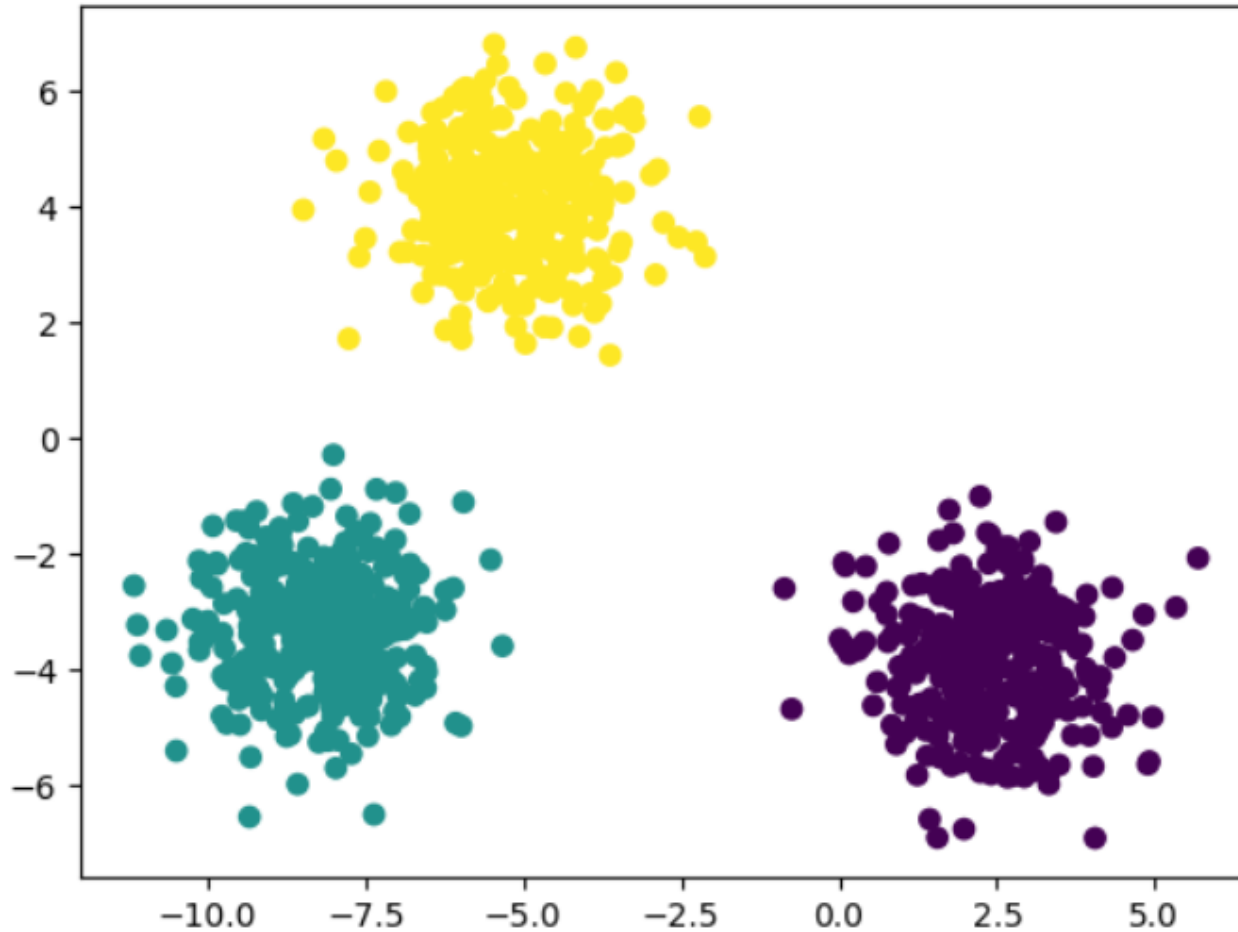
first feature (x-coordinate)

second feature (y-coordinate)

C – color for each point

```
plt.scatter(x[:,0],x[:,1],c=y)
```

```
<matplotlib.collections.PathCollection at 0x7ffa5c58e2f0>
```



y array (cluster labels) is passed, which means **points are colored** according to their **cluster label (0, 1, or 2)**.

✓
0s



#standardization-Feature scaling technique

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()
```

✓
2s

[10] Generated code may be subject to a license |

#train test split

```
from sklearn.model_selection import train_test_split  
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.33,random_state=42)
```

✓
0s



```
X_train_scaled = scaler.fit_transform(x_train) # convert the features into equivalent scale between 0 -1  
X_test_scaled = scaler.transform(x_test)
```

✓
0s

```
[16] Generated code may be subject to a license | SuRreal1000/capstone_know_your_ship
#Use Elbow method to select K -value
wcss = [] # Initialize an empty list to store Within-Cluster Sum of Squares (WCSS)
for k in range (1,11): # Loop over k values from 1 to 10
    kmeans=KMeans(n_clusters=k,init='k-means++') # KMeans with 'k-means++' initialization
    kmeans.fit(X_train_scaled)# Fit the model on the scaled data
    wcss.append(kmeans.inertia_) # Append the WCSS (inertia) to the list
```



✓
0s



WCSS



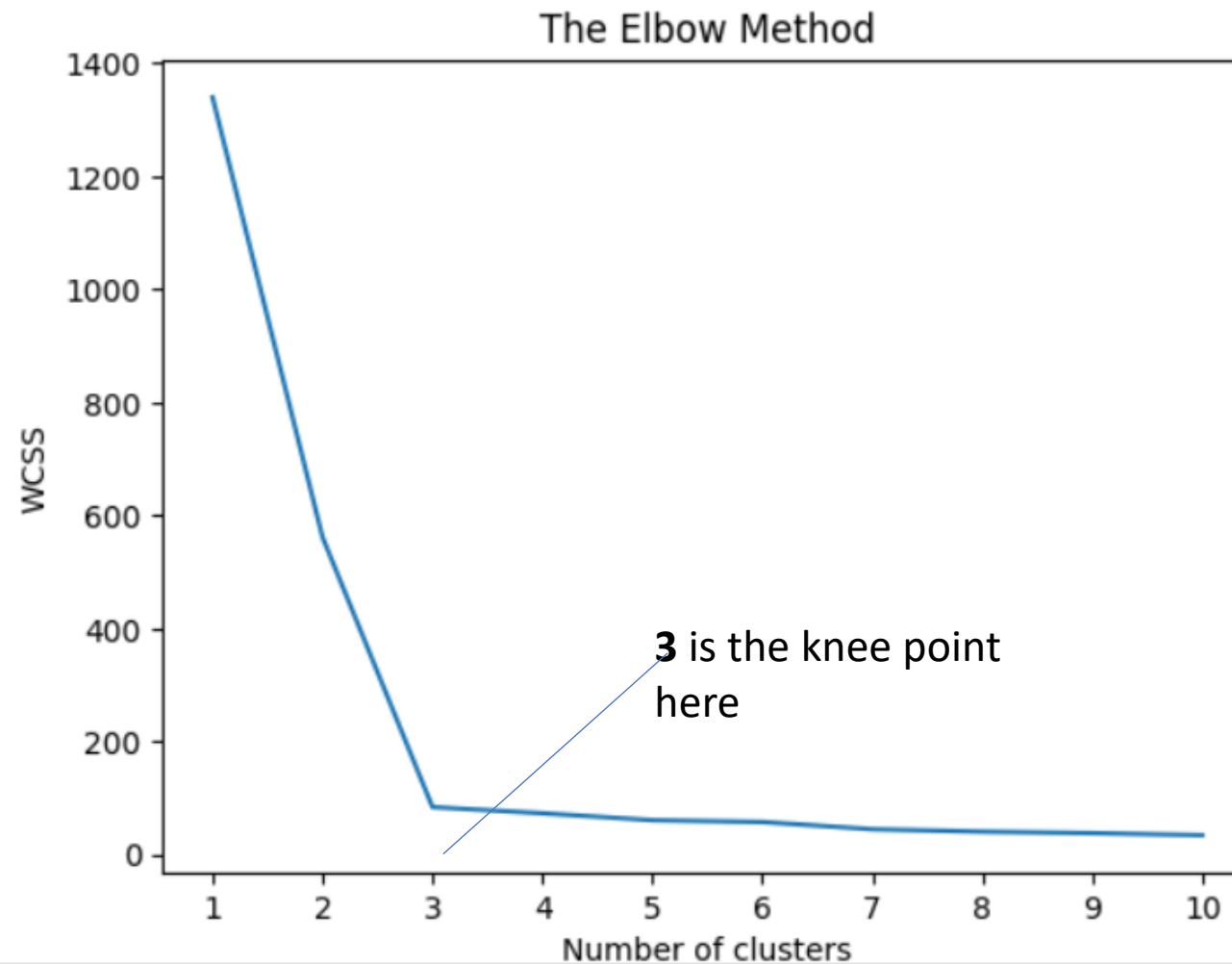
```
[1340.00000000000025,
 561.5437505414553,
 83.88246250594051,
 72.98911646656772,
 60.69296434118334,
 57.53647143955767,
 44.70737668862124,
 40.414942228573764,
 37.79420544608191,
 34.1945780226129]
```

Wcss value decreases as the **k value**
increases



#plot the elbow curve

```
plt.plot(range(1,11),wcss) # Line plot of k vs WCSS  
plt.xticks(range(1,11)) # Set x-axis ticks to match cluster numbers 1 to 10  
plt.title('The Elbow Method')  
plt.xlabel('Number of clusters')  
plt.ylabel('WCSS')  
plt.show()
```



So cluster should be **3**

↑ ↓ ✦ 🔗 💬 ⚙

[illegible]

[22] Generated code may be subject to a license | ArundatiGarikipeta/ybiinternshipprojects

```
y_pred = kmeans.predict(X_test_scaled) #For each point in X_test_scaled, the model:  
                                         #Calculates its distance to all cluster centers.  
                                         #Assigns it to the closest cluster.
```



y_pred

#contains the cluster labels for the test data.



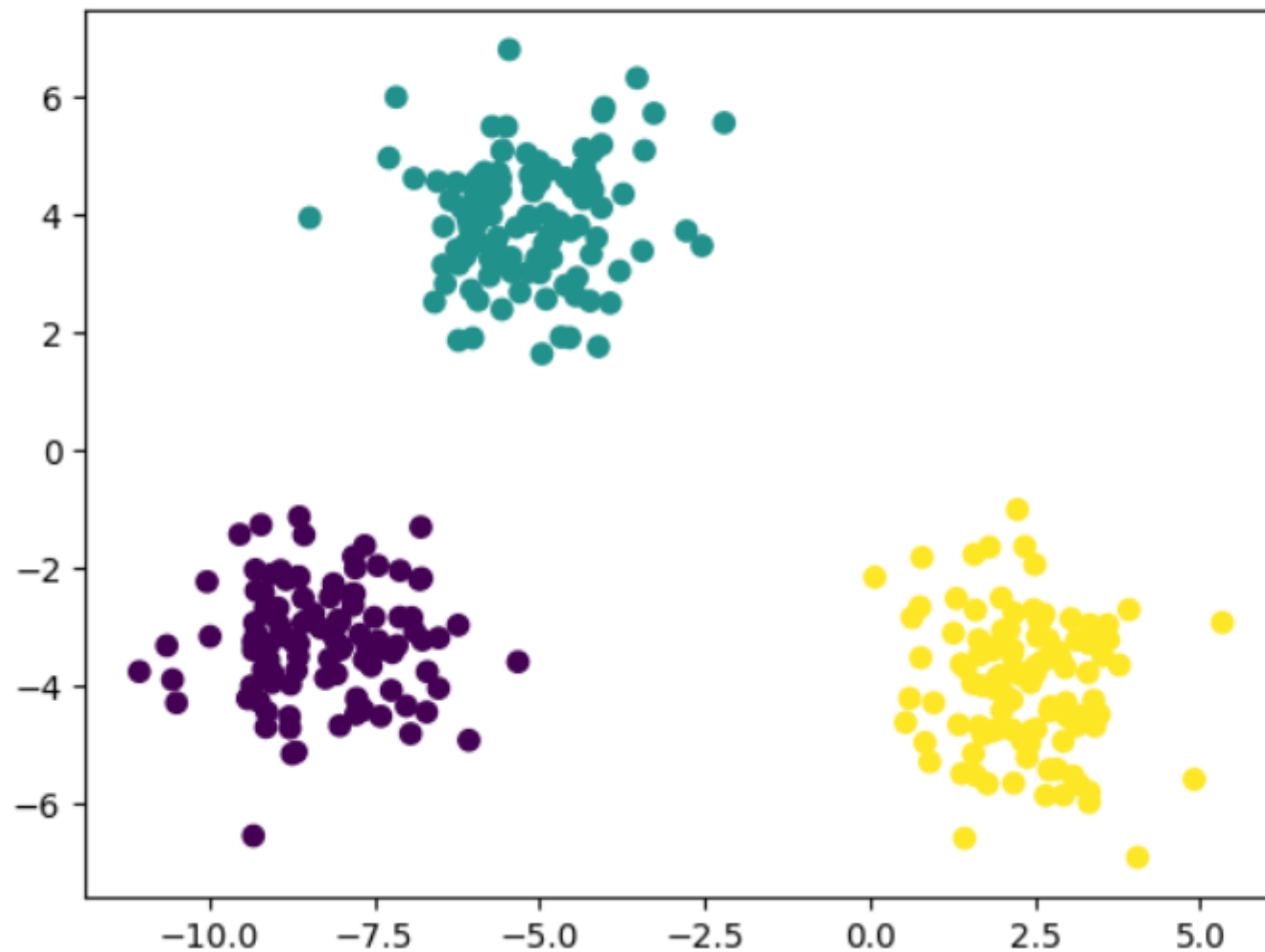
```
array([2, 1, 2, 1, 1, 0, 1, 2, 1, 2, 0, 1, 2, 1, 0, 1, 1, 0, 1, 1, 1, 2,  
       0, 1, 0, 1, 2, 1, 1, 0, 0, 0, 2, 1, 0, 1, 0, 1, 2, 0, 0, 2, 0, 2,  
       0, 1, 1, 0, 2, 1, 0, 1, 0, 0, 1, 0, 2, 0, 0, 1, 2, 2, 0, 0, 0, 2,  
       1, 2, 0, 0, 2, 2, 1, 0, 0, 2, 0, 1, 0, 2, 0, 2, 0, 0, 2, 0, 1, 2,  
       1, 1, 0, 0, 2, 0, 1, 1, 1, 0, 2, 0, 0, 1, 0, 2, 0, 1, 1, 0, 2, 0,  
       2, 0, 2, 1, 0, 2, 2, 2, 1, 2, 2, 1, 1, 1, 2, 2, 1, 1, 1, 1, 2, 0,  
       2, 2, 2, 2, 2, 1, 0, 2, 2, 1, 0, 0, 2, 0, 1, 2, 2, 2, 2, 0, 2, 2,  
       0, 2, 1, 2, 2, 2, 0, 0, 0, 2, 1, 2, 1, 1, 2, 1, 0, 2, 1, 1, 2, 0,  
       2, 1, 2, 2, 0, 2, 1, 1, 1, 0, 2, 1, 2, 2, 1, 0, 0, 1, 0, 2, 2, 2,  
       2, 0, 1, 2, 0, 0, 1, 2, 0, 1, 0, 0, 1, 1, 1, 2, 2, 1, 2, 2, 1, 1,  
       2, 1, 1, 1, 1, 1, 0, 1, 1, 0, 2, 1, 0, 2, 1, 1, 0, 2, 1, 2, 2, 0,  
       0, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 1, 0, 0, 1, 2, 2, 1, 0, 0, 1,  
       1, 1, 0, 1, 1, 2, 1, 0, 0, 2, 0, 2, 1, 0, 1, 2, 0, 1, 1, 0, 0, 1,  
       1, 1, 2, 0, 1, 2, 1, 0, 2, 2, 2, 1, 2, 1, 0, 2, 0, 0, 1, 1, 0, 1,  
       1, 2, 0, 1, 1, 1, 1, 1, 1, 1, 2, 0, 1, 2, 2, 1, 2, 0, 0, 0, 0, 1],  
      dtype=int32)
```




```
plt.scatter(x_test[:,0],x_test[:,1],c=y_pred) #color of each point based on  
#the predicted cluster labels (y_pred).
```



```
<matplotlib.collections.PathCollection at 0x7ffa4c105360>
```



```
[ ] # To validate the K value  
# 1.kneelocator  
# 2.silhouette scoring
```

```
[30] !pip install kneed # install kneed library manually
```



Collecting kneed

Downloading kneed-0.8.5-py3-none-any.whl.metadata (5.5 kB)

Requirement already satisfied: numpy>=1.14.2 in /usr/local/lib/python3.10/dist-packages (from kneed)

Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from kneed)

Downloading kneed-0.8.5-py3-none-any.whl (10 kB)

Installing collected packages: kneed

Successfully installed kneed-0.8.5



js

```
[35] from kneed import KneeLocator #A Python library that helps to automatically  
#detect the "elbow point" (or "knee") in a curve.
```



```
#No need to manually inspect the elbow point. It will automate the process  
kl=KneeLocator(range(1,11),wcss,curve='convex',direction='decreasing')
```

▶ `kl.elbow #Optimal number of clusters: <value of k>`

↔ 3

✓
0s

```
[39] #silhoutte scoring  
      from sklearn.metrics import silhouette_score
```

```
[40] # Initialize an empty list to store silhouette coefficients  
      silhouette_coefficients=[]  
      # Loop through different values of k (number of clusters) from 2 to 10  
      for k in range(2,11):  
          kmeans=KMeans(n_clusters=k,init='k-means++')  
          kmeans.fit(X_train_scaled)  
          #Stores the silhouette score for each k value (number of clusters) in the list  
          score=silhouette_score(X_train_scaled,kmeans.labels_)  
          # Append the silhouette score to the list  
          silhouette_coefficients.append(score)
```

✓
0s



silhouette_coefficients



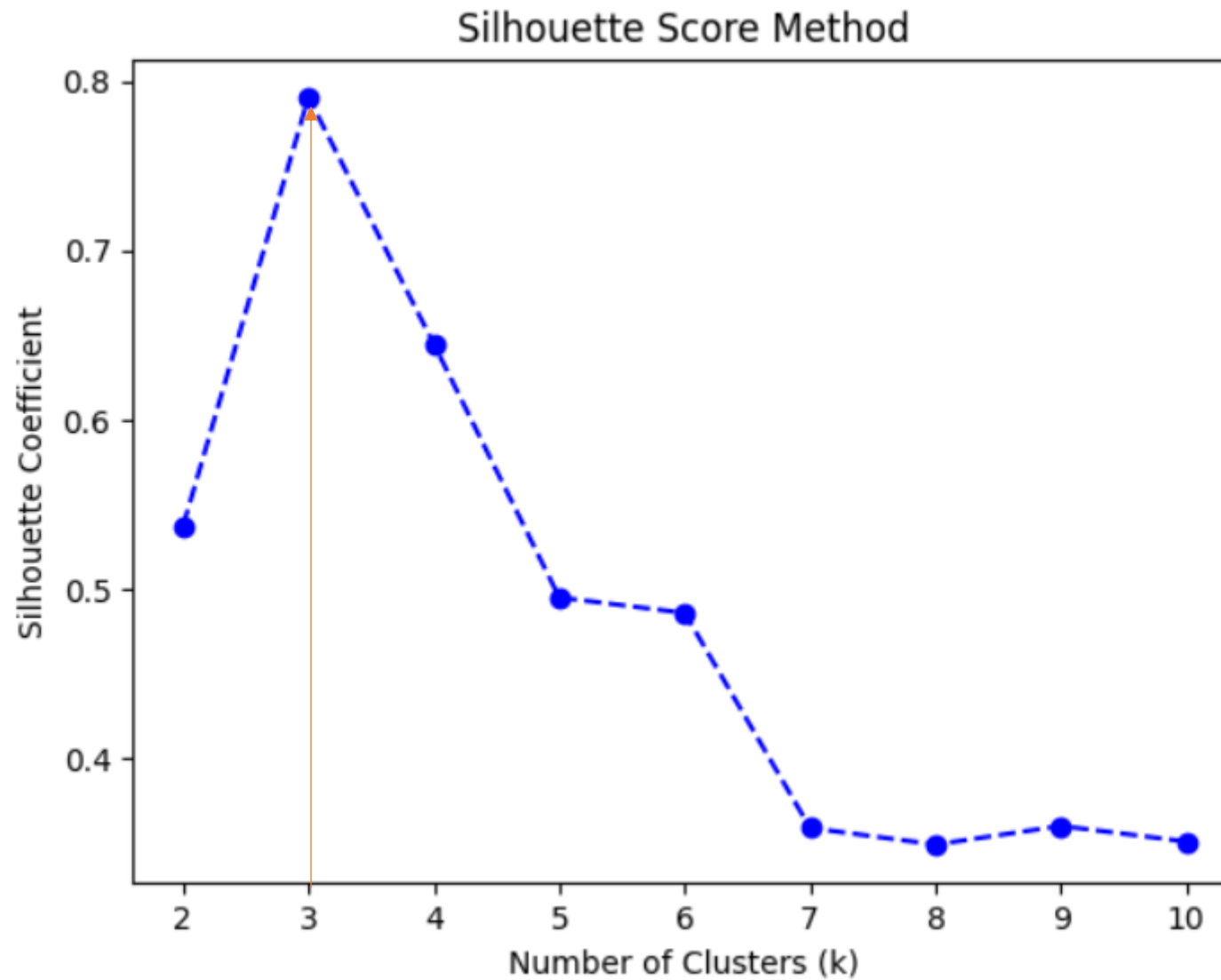
```
[0.5369254276982819,  
 0.7906053008449458,  
 0.6445893828663066,  
 0.4946994513671938,  
 0.4855360480189789,  
 0.35831179000646235,  
 0.34852212339671274,  
 0.3595337190961337,  
 0.35028805987946227]
```



✓
0s



```
# Plot silhouette scores for different k values  
plt.plot(range(2, 11), silhouette_coefficients, marker='o', linestyle='--', color='b')  
plt.title('Silhouette Score Method')  
plt.xlabel('Number of Clusters (k)')  
plt.ylabel('Silhouette Coefficient')  
plt.show()
```



Since silhouette score is high in **k value 3**, we can **prefer k = 3 (clusters)**

Hierarchical Clustering Implementation

✓
3s

```
[2] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
```

✓
0s

```
[3] #Import IRIS datasets
iris = datasets.load_iris()
```

✓
0s



iris



```
{'data': array([[5.1, 3.5, 1.4, 0.2],
                [4.9, 3. , 1.4, 0.2],
                [4.7, 3.2, 1.3, 0.2],
                [4.6, 3.1, 1.5, 0.2],
                [5. , 3.6, 1.4, 0.2],
                [5.4, 3.9, 1.7, 0.4],
                [4.6, 3.4, 1.4, 0.3],
                [5. , 3.4, 1.5, 0.2],
                [4.4, 2.9, 1.4, 0.2],
                [4.9, 3.1, 1.5, 0.1],
                [5.4, 3.7, 1.5, 0.2],
                [4.8, 3.4, 1.6, 0.2],
                [4.8, 3. , 1.4, 0.1],
                [4.3, 3. , 1.1, 0.1],
                [5.8, 4. , 1.2, 0.2],
                [5.7, 4.4, 1.5, 0.4],
                [5.4, 3.9, 1.3, 0.4],
                [5.1, 3.5, 1.4, 0.3],
                [5.7, 3.8, 1.7, 0.3],
```

```
[6] iris_data = pd.DataFrame(iris.data) # Create a iris data frame
```

✓
0s



```
iris_data.head()
```



	0	1	2	3
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2



Next steps:

[Generate code with iris_data](#)[View recommended plots](#)[New interactive sheet](#)


```
iris_data.columns = iris.feature_names # To give column names to the iris data frame
```

✓
0s

```
iris_data
```



	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8



✓
0s [10] #Preprocessing
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

✓
0s [11] Generated code may be subject to a license | IBM-EPBL/IBM-Project-27042-1660044679
x_scale = scaler.fit_transform(iris_data)

✓
0s  x_scale

 array([[-9.00681170e-01, 1.01900435e+00, -1.34022653e+00,
 -1.31544430e+00],
 [-1.14301691e+00, -1.31979479e-01, -1.34022653e+00,
 -1.31544430e+00],
 [-1.38535265e+00, 3.28414053e-01, -1.39706395e+00,
 -1.31544430e+00],
 [-1.50652052e+00, 9.82172869e-02, -1.28338910e+00,
 -1.31544430e+00],
 [-1.02184904e+00, 1.24920112e+00, -1.34022653e+00,
 -1.31544430e+00],
 [-5.37177559e-01, 1.93979142e+00, -1.16971425e+00,
 -1.05217993e+00],
 [-1.50652052e+00, 7.88807586e-01, -1.34022653e+00,
 -1.18381211e+00],
 [-1.02184904e+00, 7.88807586e-01, -1.28338910e+00,
 -1.31544430e+00],
 [-1.74885626e+00, -3.62176246e-01, -1.34022653e+00,
 -1.31544430e+00],
 [-1.14301691e+00, 9.82172869e-02, -1.28338910e+00,
 -1.44707648e+00],
 [-5.37177559e-01, 1.47939788e+00, -1.28338910e+00,

```
✓  
0s [13] #Apply the PCA to reduce the dimensions  
from sklearn.decomposition import PCA  
pca = PCA(n_components=2)# convert the four dimensions into 2 dimension
```

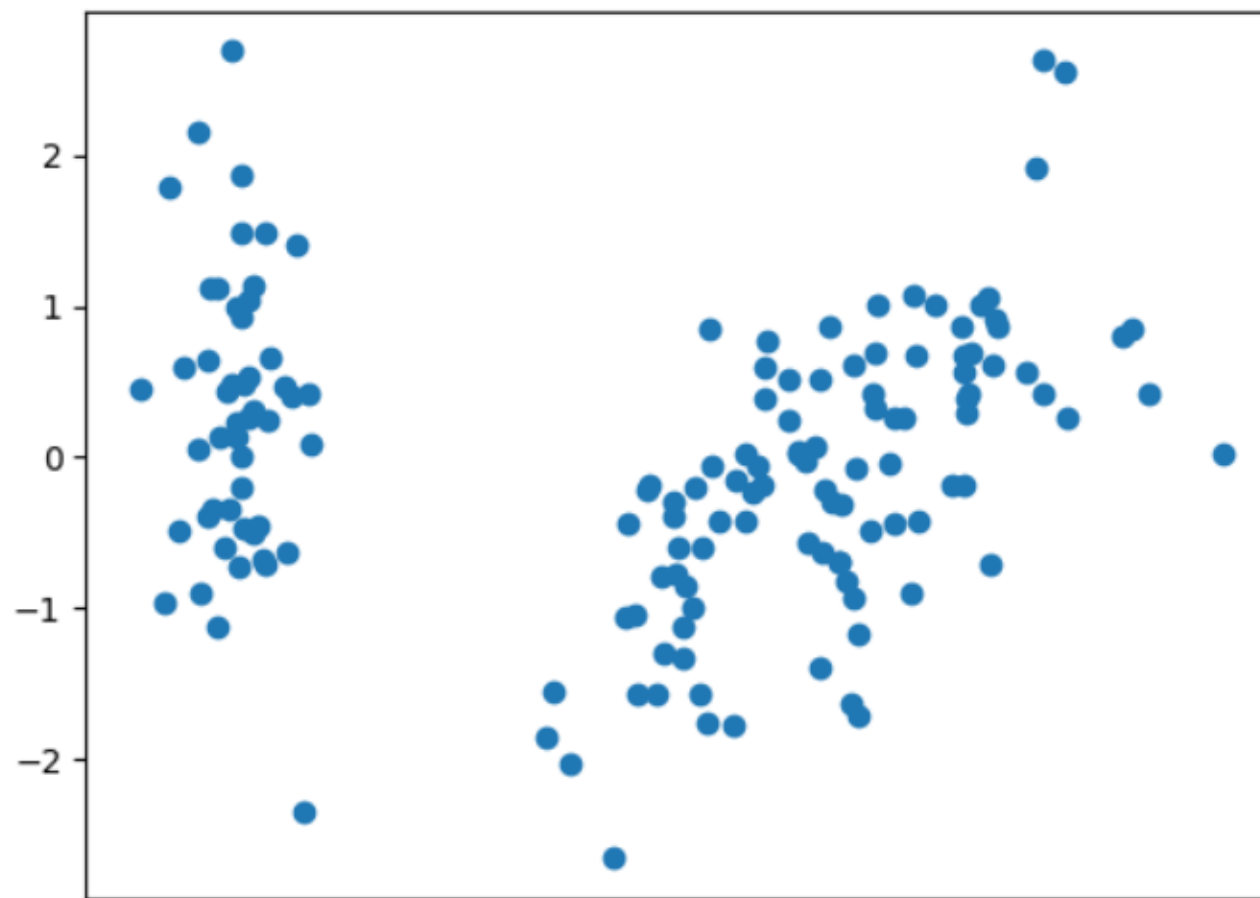
```
✓  
0s [15] pca_scaled = pca.fit_transform(x_scale)
```

x-coordinate for each data point.

y-coordinate for each data point

```
✓  
0s ▶ plt.scatter(pca_scaled[:,0],pca_scaled[:,1]) # all the rows in 0th column and 1 stcolumn plotted
```

⇒ <matplotlib.collections.PathCollection at 0x78f005d9e230>



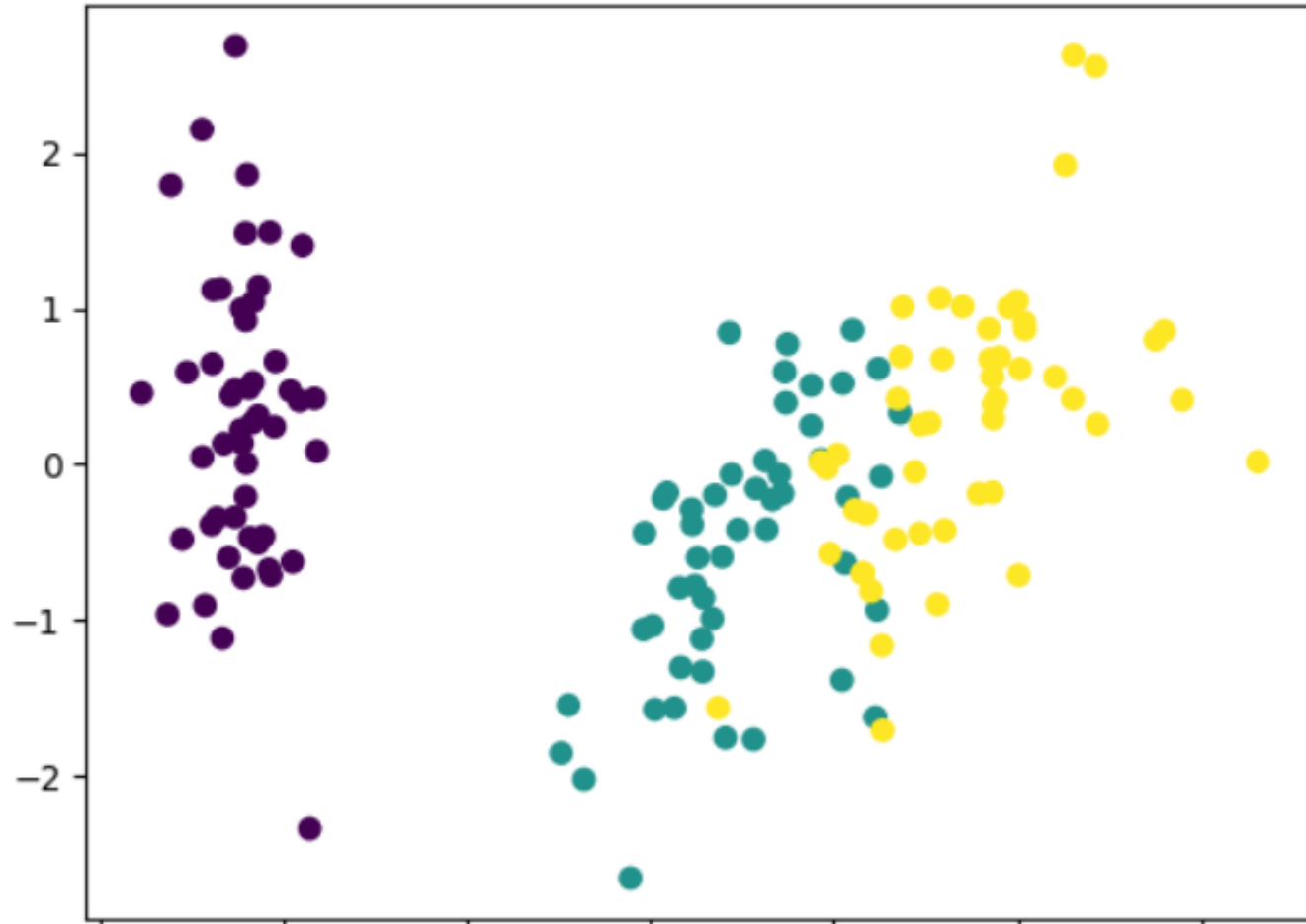
✓
0s



```
plt.scatter(pca_scaled[:,0],pca_scaled[:,1],c=iris.target) # iris.target is the class label having 3 labels
```



```
<matplotlib.collections.PathCollection at 0x78f00454a410>
```





Generated code may be subject to a license | [amirothman/scikit-learn-course](#) | [Egemenatilla/Machine-Learning-Essentials](#)

#Agglomerative clustering

To construct a dendrogram

import scipy.cluster.hierarchy as sch # hierarchical or agglomerative clustering.

#plot the dendrogram

plt.figure(figsize=(10,7))

plt.title('Dendrogram')

#create the dendrogram

dendrogram = sch.dendrogram(sch.linkage(pca_scaled,method='ward'))

plt.title('Dendrogram')

plt.xlabel('sample')

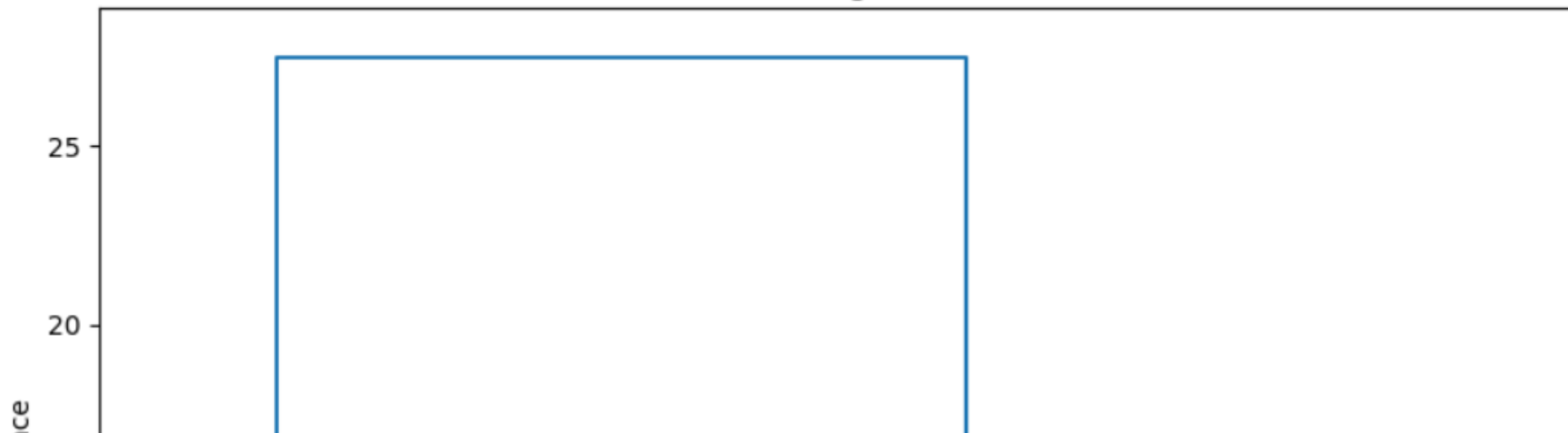
plt.ylabel('Euclidean distance')

plt.show()

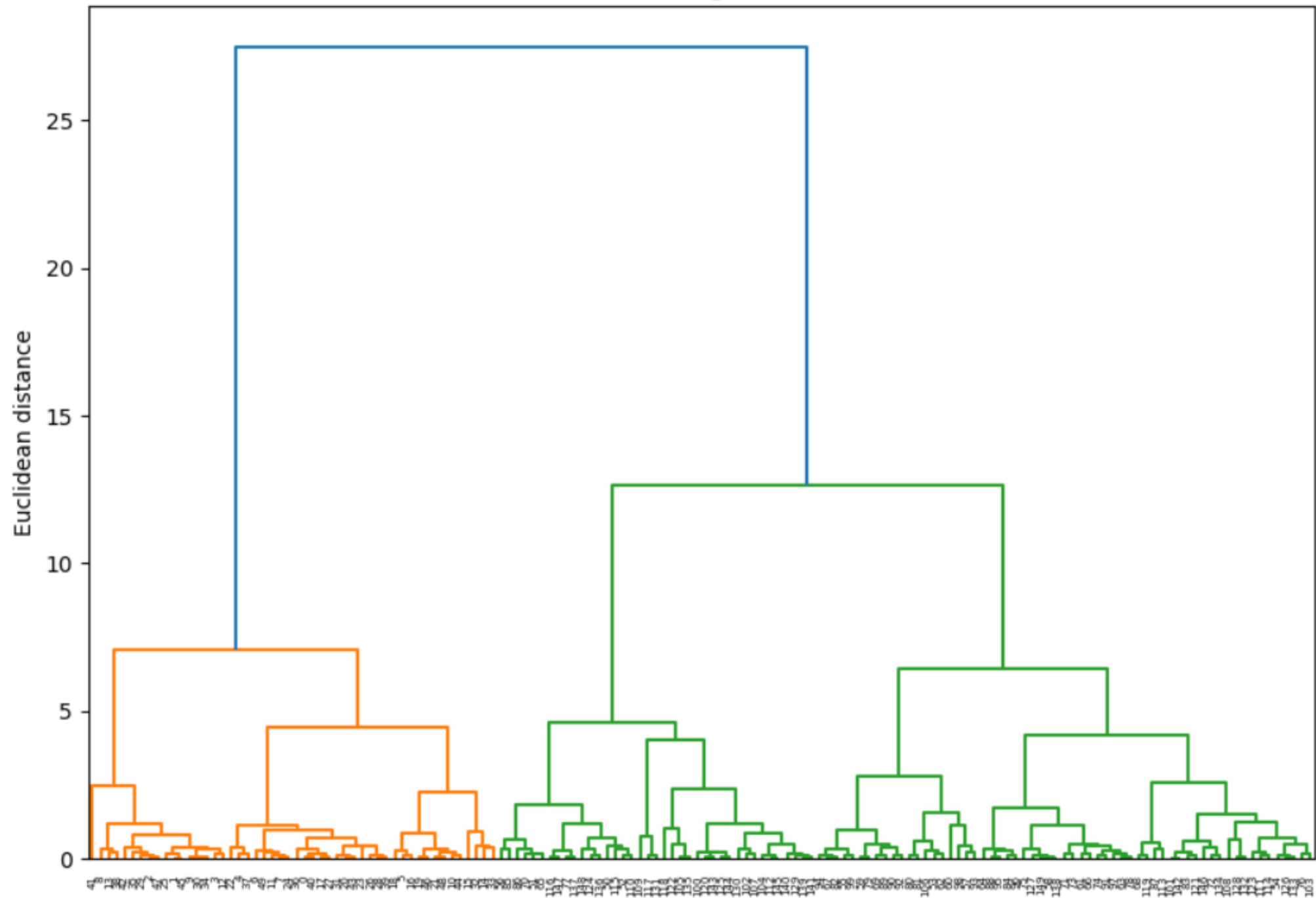
Used to generate a **dendrogram**, a **tree-like diagram** showing the **hierarchy of clusters**.



Dendrogram



Dendrogram



```
[24] from sklearn.cluster import AgglomerativeClustering #implements hierarchical agglomerative clustering.  
#ward - Merges clusters by minimizing the variance within the clusters.  
hc = AgglomerativeClustering(n_clusters=2, linkage='ward')  
hc.fit(pca_scaled)
```

AgglomerativeClustering

```
# Access the cluster labels after fitting the model
hc.labels      #Each number in the array represents the cluster to which a specific data point belongs.
```

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
       1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

✓
0s

```
[27] plt.scatter(pca_scaled[:,0],pca_scaled[:,1],c=hc.labels_)
```



<matplotlib.collections.PathCollection at 0x78f0019f8250>

